



PIMPRI CHINCHWAD EDUCATION TRUST'S.
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(An Autonomous Institute)

B. TECH

Year: 2026 – 27

Semester: VII

Name of the Student: Swayam Mandhani

PRN: 123B1B184

Department: Computer Engineering

Division: C, **Batch** – C2

Course: DevOps

Course Code: BCE27PE01

Assignment No – 7

Title:

Explore Kubernetes Container Orchestration and Deploy an Application Using Helm.

Aim:

To study Kubernetes architecture and understand the structure and commands of Helm for managing Kubernetes applications.

Objectives:

- Understand container orchestration.
- Study Kubernetes architecture.
- Identify Kubernetes control-plane and worker-node components.
- Understand Helm and Helm charts.
- Create a basic Helm chart.
- Perform Helm install, upgrade, template, and uninstall operations.
- Verify deployed Pods, Deployments, and Services using kubectl.

Theory:

Kubernetes Container Orchestration

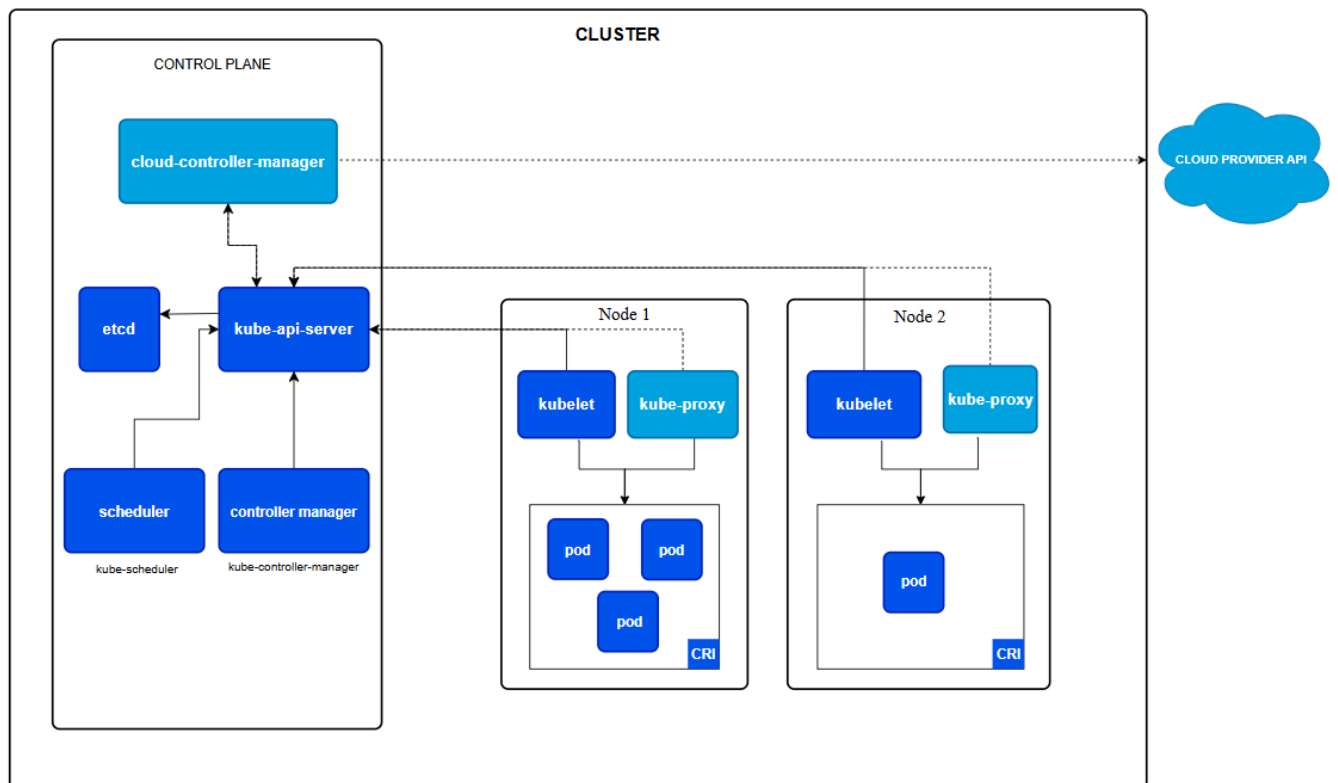
Kubernetes is a container orchestration platform used to automate the deployment, scaling, networking, service discovery, load balancing, and recovery of containerized applications. It manages application workloads as Pods and maintains the desired state of resources in a cluster.

Kubernetes Architecture

A Kubernetes cluster consists of a control plane and one or more worker nodes. The control plane coordinates cluster operations, while worker nodes run application Pods.

Component	Purpose
API Server	Entry point for Kubernetes API requests.
Etd	Stores cluster configuration and state.
Scheduler	Assigns Pods to suitable worker nodes.
Controller Manager	Maintains the desired cluster state.
Kubelet	Manages Pods and containers on worker nodes.
Kube-proxy	Provides networking and service connectivity.
Container Runtime	Executes containers on worker nodes.

Kubernetes Architecture Diagram



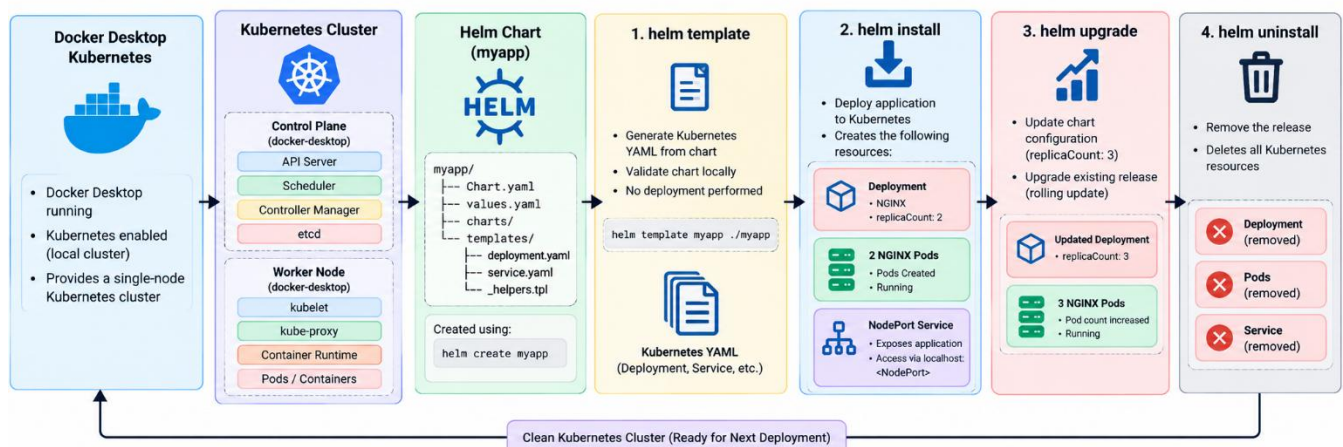
Helm

Helm is a package manager for Kubernetes. It simplifies deployment and management of Kubernetes applications through reusable packages known as Helm Charts.

Helm Chart Components

- Chart.yaml — contains chart metadata such as name, version, type, and application version.
- values.yaml — contains configurable values used by templates.
- templates/ — contains Kubernetes resource templates such as Deployment and Service.
- charts/ — stores chart dependencies when required.

System / Practical Workflow:



Prerequisites and Tools:

- Docker Desktop with Kubernetes enabled.
- kubectl command-line tool.
- Helm command-line tool.
- Windows PowerShell.
- Web browser for application verification.
- Local Kubernetes cluster using the Docker Desktop context.

Code / Implementation:

Project Structure

Assignment-7/

```
└─ myapp/
    ├── Chart.yaml
    ├── values.yaml
    ├── charts/
    └─ templates/
        ├── deployment.yaml
        ├── service.yaml
        └─ _helpers.tpl
```

Chart.yaml

```
apiVersion: v2
name: myapp
description: A Helm chart for Kubernetes
type: application
version: 0.1.0
appVersion: "1.0"
```

Important values.yaml Configuration

```
replicaCount: 2

image:
  repository: nginx
  tag: "latest"

service:
  type: NodePort
  port: 80
```

Kubernetes and Helm Commands

```
# Kubernetes verification
docker --version
```

```
kubectl version --client
```

```
kubectl config current-context
```

```
kubectl cluster-info
```

```
kubectl get nodes
```

```
kubectl get pods -A
```

```
kubectl get namespaces
```

```
# Helm verification
```

```
helm version
```

```
# Create and validate chart
```

```
helm create myapp
```

```
helm lint .\myapp
```

```
# Render Kubernetes YAML without deploying the resources to the cluster.
```

```
helm template myapp .\myapp
```

```
# Install and inspect release
```

```
helm install myrelease .\myapp
```

```
helm list
```

```
helm status myrelease
```

```
# Kubernetes verification
```

```
kubectl get pods
```

```
kubectl get deployments
```

```
kubectl get services
```

```
# Upgrade
```

```
helm upgrade myrelease .\myapp
```

```
helm status myrelease
```

```
kubectl get pods
```

```
# Uninstall
```

```
helm uninstall myrelease
```

```
helm list
```

```
kubectl get pods
```

```
kubectl get deployments
```

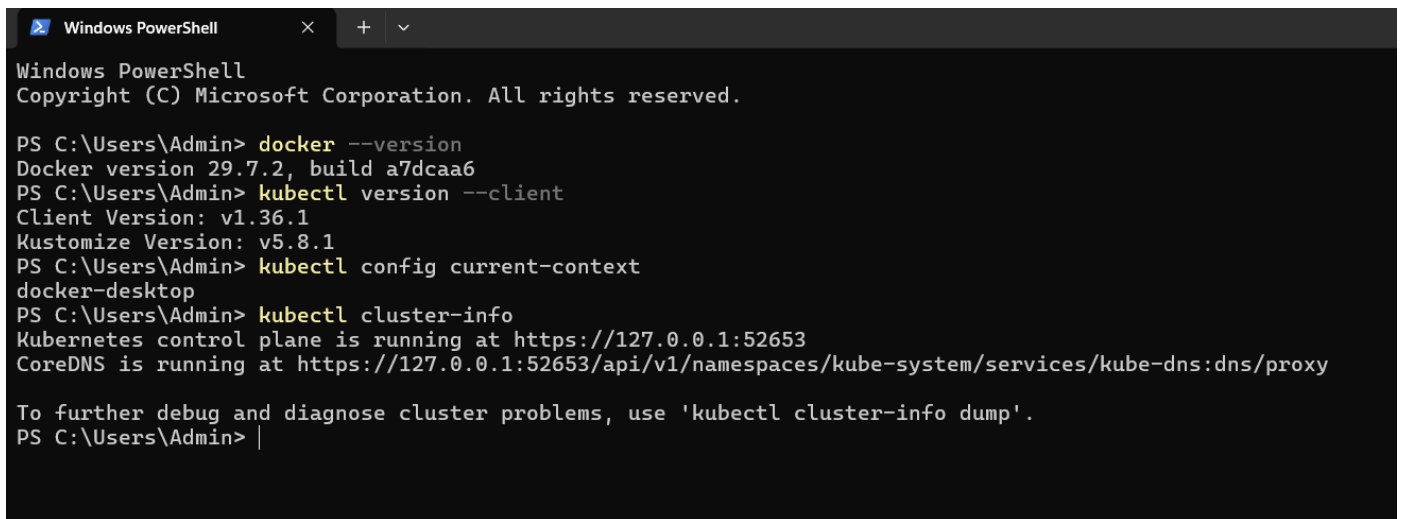
```
kubectl get services
```

Application Access

The Helm chart deployed an NGINX application through a NodePort Service. The Kubernetes Service was observed with port mapping 80:32353/TCP. Because local port 8080 was unavailable on Windows, an alternative local port-forward was used to verify the NGINX application in the browser.

Output

The following figures provide evidence of Kubernetes cluster verification, Helm installation and chart creation, template rendering, deployment, Kubernetes resource verification, application access, Helm upgrade, and release removal.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Admin> docker --version
Docker version 29.7.2, build a7dcaa6
PS C:\Users\Admin> kubectl version --client
Client Version: v1.36.1
Kustomize Version: v5.8.1
PS C:\Users\Admin> kubectl config current-context
docker-desktop
PS C:\Users\Admin> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:52653
CoreDNS is running at https://127.0.0.1:52653/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\Admin> |
```

Figure 1: Kubernetes Tools Verification

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Admin> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:52653
CoreDNS is running at https://127.0.0.1:52653/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\Users\Admin> kubectl get namespaces
NAME                STATUS    AGE
default             Active   3m52s
kube-node-lease     Active   3m52s
kube-public         Active   3m52s
kube-system         Active   3m52s
local-path-storage  Active   3m45s
PS C:\Users\Admin> |
```

Figure 2: Kubernetes Cluster Information

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Admin> kubectl get nodes
NAME                STATUS    ROLES    AGE    VERSION
desktop-control-plane Ready    control-plane 4m9s   v1.36.1
PS C:\Users\Admin> |
```

Figure 3: Kubernetes Nodes

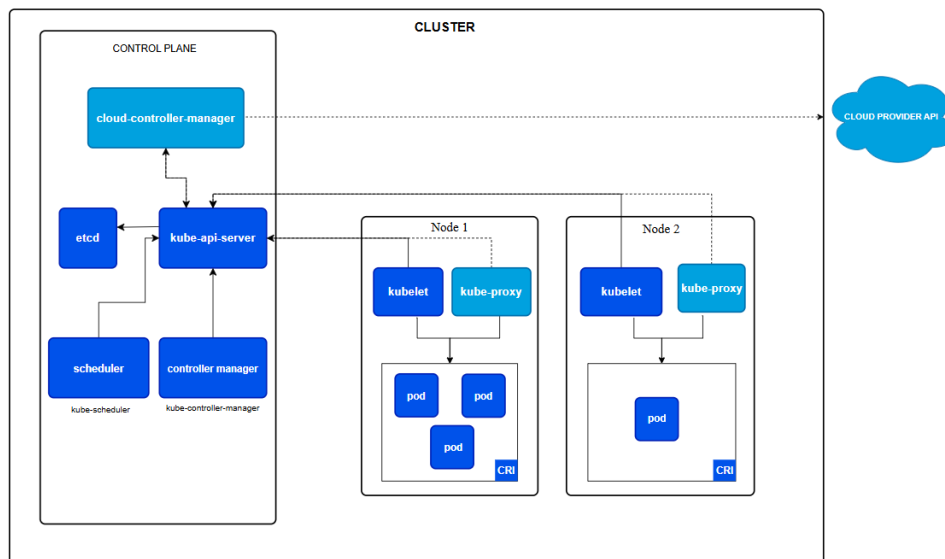


Figure 4: Kubernetes Architecture

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> helm version
version.BuildInfo{Version:"v4.3.0", GitCommit:"bec5b06ed841fe5269972d864d5177944fd5970f", GitTreeState:"clean", GoVersion:"go1.27.1", KubeClientVersion:"v1.37"}
PS C:\WINDOWS\system32> helm help
The Kubernetes package manager

Common actions for Helm:

- helm search:    search for charts
- helm pull:      download a chart to your local directory to view
- helm install:   upload the chart to Kubernetes
- helm list:      list releases of charts

Environment variables:
```

Name	Description
\$HELM_CACHE_HOME	set an alternative location for storing cached files.
\$HELM_CONFIG_HOME	set an alternative location for storing Helm configuration.
\$HELM_DATA_HOME	set an alternative location for storing Helm data.
\$HELM_DEBUG	indicate whether or not Helm is running in Debug mode
\$HELM_DRIVER	set the backend storage driver. Values are: configmap, secret, memory, sql.
\$HELM_DRIVER_SQL_CONNECTION_STRING	set the connection string the SQL storage driver should use.
\$HELM_MAX_HISTORY	set the maximum number of helm release history.
\$HELM_NAMESPACE	set the namespace used for the helm operations.
\$HELM_NO_PLUGINS	disable plugins. Set HELM_NO_PLUGINS=1 to disable plugins.
\$HELM_PLUGINS	set the path to the plugins directory
\$HELM_REGISTRY_CONFIG	set the path to the registry config file.
\$HELM_REPOSITORY_CACHE	set the path to the repository cache directory
\$HELM_REPOSITORY_CONFIG	set the path to the repositories file.
\$KUBECONFIG	set an alternative Kubernetes configuration file (default ~/.kube/config)
\$HELM_KUBEAPISERVER	set the Kubernetes API Server Endpoint for authentication
\$HELM_KUBECAFILE	set the Kubernetes certificate authority file.
\$HELM_KUBEASGROUPS	set the Groups to use for impersonation using a comma-separated list.
\$HELM_KUBEASUSER	set the Username to impersonate for the operation.
\$HELM_KUBECONTEXT	set the name of the kubeconfig context.
\$HELM_KUBETOKEN	set the Bearer KubeToken used for authentication.
\$HELM_KUBEINSECURE_SKIP_TLS_VERIFY	indicate if the Kubernetes API server's certificate validation should be skipped (insecure)
\$HELM_KUBETLS_SERVER_NAME	set the server name used to validate the Kubernetes API server certificate
\$HELM_BURST_LIMIT	set the default burst limit in the case the server contains many CRDs (default 100, -1 to disable)
\$HELM_QPS	set the Queries Per Second in cases where a high number of calls exceed the option for higher burst values
\$HELM_COLOR	set color output mode. Allowed values: never, always, auto (default: never)
\$NO_COLOR	set to any non-empty value to disable all colored output (overrides \$HELM_COLOR)
\$SOURCE_DATE_EPOCH	set a Unix timestamp for reproducible chart archives

Figure 5: Helm Version Verification

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\Projects\DevOps Github\Assignment-7> helm create myapp
Creating myapp
PS D:\Projects\DevOps Github\Assignment-7> Get-ChildItem .\myapp

Directory: D:\Projects\DevOps Github\Assignment-7\myapp

Mode                LastWriteTime         Length Name
----                -
d-----          9/30/2026  10:38 AM             charts
d-----          9/30/2026  10:38 AM             templates
-a----          9/30/2026  10:38 AM           349 .helmignore
-a----          9/30/2026  10:38 AM          1141 Chart.yaml
-a----          9/30/2026  10:38 AM          5271 values.yaml
```

Figure 6: Helm Chart Creation

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\Projects\DevOps Github\Assignment-7> tree .\myapp /F
Folder PATH listing for volume New Volume
Volume serial number is 08E1-6123
D:\PROJECTS\DEVOPS GITHUB\ASSIGNMENT-7\MYAPP
|
| .helmignore
| Chart.yaml
| values.yaml
|
|---charts
|
|---templates
|    deployment.yaml
|    hpa.yaml
|    httproute.yaml
|    ingress.yaml
|    NOTES.txt
|    service.yaml
|    serviceaccount.yaml
|    _helpers.tpl
|
|---tests
|    test-connection.yaml
```

Figure 7: Helm Chart Structure

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\Projects\DevOps Github\Assignment-7> helm lint .\myapp
==> Linting .\myapp
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed
PS D:\Projects\DevOps Github\Assignment-7> helm template myapp ./myapp
---
# Source: myapp/templates/serviceaccount.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: myapp
  labels:
    helm.sh/chart: myapp-0.1.0
    app.kubernetes.io/name: myapp
    app.kubernetes.io/instance: myapp
    app.kubernetes.io/version: "1.16.0"
    app.kubernetes.io/managed-by: Helm
  automountServiceAccountToken: true
---
# Source: myapp/templates/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp
  labels:
    helm.sh/chart: myapp-0.1.0
    app.kubernetes.io/name: myapp
    app.kubernetes.io/instance: myapp
    app.kubernetes.io/version: "1.16.0"
    app.kubernetes.io/managed-by: Helm
spec:
  type: NodePort
  ports:
    - port: 80
      targetPort: http
      protocol: TCP
      name: http
  selector:
    app.kubernetes.io/name: myapp
    app.kubernetes.io/instance: myapp
---
# Source: myapp/templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  labels:
    helm.sh/chart: myapp-0.1.0
    app.kubernetes.io/name: myapp
    app.kubernetes.io/instance: myapp
    app.kubernetes.io/version: "1.16.0"
    app.kubernetes.io/managed-by: Helm
spec:
  replicas: 2
  selector:
    matchLabels:
      app.kubernetes.io/name: myapp
```

Figure 8: Helm Template Output

```

Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-7> helm install myrelease ./myapp
NAME: myrelease
LAST DEPLOYED: Wed Sep 30 10:45:28 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myrelease-myapp)
  export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT
PS D:\Projects\DevOps Github\Assignment-7> |

```

Figure 9: Helm Install Success

```

Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-7> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myrelease-myapp-5fccf97548-2k6gr    1/1     Running   0           97s
myrelease-myapp-5fccf97548-lwc48    1/1     Running   0           97s
PS D:\Projects\DevOps Github\Assignment-7> kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
myrelease-myapp    2/2     2             2           101s
PS D:\Projects\DevOps Github\Assignment-7> kubectl get services
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1     <none>        443/TCP          17m
myrelease-myapp     NodePort    10.96.74.255  <none>        80:32353/TCP     104s
PS D:\Projects\DevOps Github\Assignment-7> |

```

Figure 10: Kubernetes Pods, Deployment and Service

```

Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-7> helm list
NAME      NAMESPACE   REVISION   UPDATED                               STATUS   CHART       APP VERSION
myrelease default     1          2026-09-30 10:45:28.3636461 +0530 IST deployed myapp-0.1.0 1.16.0
PS D:\Projects\DevOps Github\Assignment-7> helm status myrelease
NAME: myrelease
LAST DEPLOYED: Wed Sep 30 10:45:28 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
RESOURCES:
==> v1/ServiceAccount
NAME      AGE
myrelease-myapp  2m46s

==> v1/Service
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)          AGE
myrelease-myapp     NodePort    10.96.74.255  <none>        80:32353/TCP     2m46s

==> v1/Deployment
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
myrelease-myapp    2/2     2             2           2m46s

==> v1/Pod(related)
NAME                                READY   STATUS    RESTARTS   AGE
myrelease-myapp-5fccf97548-2k6gr    1/1     Running   0           2m46s
myrelease-myapp-5fccf97548-lwc48    1/1     Running   0           2m46s

NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myrelease-myapp)
  export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT
PS D:\Projects\DevOps Github\Assignment-7> |

```

Figure 11: Helm List and Release Status

```
Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-7> kubectl get service
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1     <none>       443/TCP          21m
myrelease-myapp      NodePort    10.96.74.255  <none>       80:32353/TCP     5m41s
PS D:\Projects\DevOps Github\Assignment-7> kubectl port-forward service/myrelease-myapp 8081:80
Forwarding from 127.0.0.1:8081 -> 80
Forwarding from [::1]:8081 -> 80
Handling connection for 8081
Handling connection for 8081
```

Figure 12: Kubernetes Port-Forward Configuration

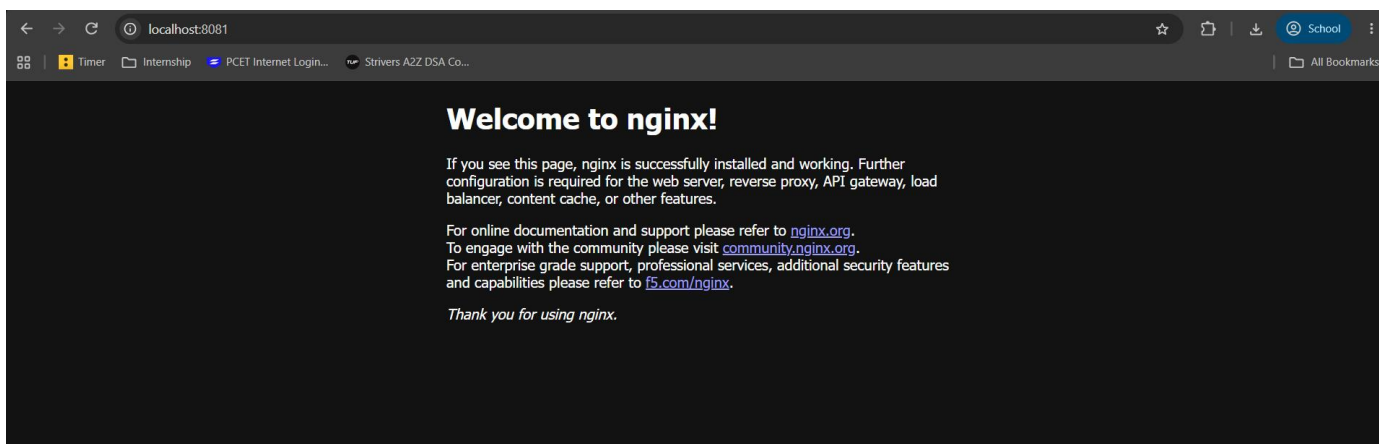


Figure 13: Kubernetes Application Running

```
Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-7> helm upgrade myrelease .\myapp
Release "myrelease" has been upgraded. Happy Helming!
NAME: myrelease
LAST DEPLOYED: Wed Sep 30 10:56:58 2026
NAMESPACE: default
STATUS: deployed
REVISION: 4
DESCRIPTION: Upgrade complete
NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myrelease-myapp)
  export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT
PS D:\Projects\DevOps Github\Assignment-7> |
```

Figure 14: Helm Upgrade

```
Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-7> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myrelease-myapp-5fccf97548-2k6gr    1/1     Running   0           12m
myrelease-myapp-5fccf97548-6tqf2    1/1     Running   0           50s
myrelease-myapp-5fccf97548-lwc48    1/1     Running   0           12m
PS D:\Projects\DevOps Github\Assignment-7> helm status myrelease
NAME: myrelease
LAST DEPLOYED: Wed Sep 30 10:56:58 2026
NAMESPACE: default
STATUS: deployed
REVISION: 4
DESCRIPTION: Upgrade complete
RESOURCES:
==> v1/ServiceAccount
NAME      AGE
myrelease-myapp  12m

==> v1/Service
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
myrelease-myapp  NodePort    10.96.74.255  <none>         80:32353/TCP     12m

==> v1/Deployment
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
myrelease-myapp  3/3     3            3           12m

==> v1/Pod(related)
NAME                                READY   STATUS    RESTARTS   AGE
myrelease-myapp-5fccf97548-2k6gr    1/1     Running   0           12m
myrelease-myapp-5fccf97548-6tqf2    1/1     Running   0           55s
myrelease-myapp-5fccf97548-lwc48    1/1     Running   0           12m

NOTES:
1. Get the application URL by running these commands:
  export NODE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].nodePort}" services myrelease-myapp)
  export NODE_IP=$(kubectl get nodes --namespace default -o jsonpath="{.items[0].status.addresses[0].address}")
  echo http://$NODE_IP:$NODE_PORT
PS D:\Projects\DevOps Github\Assignment-7> |
```

Figure 15: Kubernetes Upgrade Verification

```
Windows PowerShell
PS D:\Projects\DevOps Github\Assignment-7> helm uninstall myrelease
release "myrelease" uninstalled
PS D:\Projects\DevOps Github\Assignment-7> helm list
NAME      NAMESPACE    REVISION    UPDATED     STATUS    CHART    APP VERSION
No resources found
PS D:\Projects\DevOps Github\Assignment-7> kubectl get pods
No resources found in default namespace.
PS D:\Projects\DevOps Github\Assignment-7> kubectl get deployments
No resources found in default namespace.
PS D:\Projects\DevOps Github\Assignment-7> kubectl get services
NAME      TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes  ClusterIP    10.96.0.1     <none>         443/TCP          28m
PS D:\Projects\DevOps Github\Assignment-7> |
```

Figure 16: Helm Uninstall

Result:

Kubernetes was successfully configured using the Docker Desktop local cluster. The Kubernetes architecture and major control-plane and worker-node components were studied. A Helm chart named myapp was created, validated, rendered using helm template, installed as the myrelease release, and verified through Pods, Deployment, and NodePort Service resources. The NGINX application was accessed successfully through the deployed Kubernetes Service. The release was upgraded by changing the replica count and verified at the updated Helm revision. Finally, the Helm release was uninstalled and the associated Kubernetes resources were verified as removed.

Learning Outcome:

The practical provided hands-on understanding of Kubernetes container orchestration, cluster architecture, Pods, Deployments, Services, and Helm-based application management. The workflow demonstrated how Helm charts can package Kubernetes resources and how applications can be rendered, installed, upgraded, inspected, and uninstalled.

Conclusion:

In this assignment, Kubernetes architecture and Helm-based application deployment were implemented using a local Docker Desktop Kubernetes cluster. The practical covered cluster verification, control-plane and worker-node concepts, Helm chart creation, configurable values, template rendering, installation, release inspection, Kubernetes resource verification, application access, upgrade, and uninstall operations. This demonstrated the basic lifecycle of managing a Kubernetes application using Helm.

References:

1. Kubernetes Documentation — <https://kubernetes.io/docs/>
2. Kubernetes kubectl Documentation — <https://kubernetes.io/docs/reference/kubectl/>
3. Helm Documentation — <https://helm.sh/docs/>
4. Helm Command Reference — <https://helm.sh/docs/helm/>

Submission Links:

GitHub Repository Link: <https://github.com/SwayamMandhani06/DevOps/tree/main/Assignment-7>

Hosted / Output Link: NA – Kubernetes and Helm practical executed locally using Docker Desktop.